Case 2 - Final Solution & Execution Plan

PreFlight: conversational shipment booking and pre-flight error checker
Ops Hackathon · Option A, all review fixes folded in · prepared 25 October 2026

The whole thing on one page

What we are building. A web app where a seller either describes a shipment in plain language or uploads the Bookings and Items sheets. Either way they get one Booking Card showing: a verdict (Blocked / Ship with care / Clear), every broken rule with severity and the fix, a recommended carrier with the reason, a plain-English confirmation, and what happens next.

The one design principle. Deterministic truth, AI voice. Every number comes from code we can test. Gemini only (a) turns the seller's sentences into a small patch of fields, (b) flags implied dangerous goods, and (c) writes explanations. If Gemini disappears, the app still works.

What judges score	Weight	Where we win it
Problem breakdown & assumptions	20%	Slides 2-4: five sub-problems, trust model, on-screen Assumptions panel
Adoption rollout	15%	Slides 6-7: forwarder-ops pilot, "what happens when the tool is wrong", usage metric
MVP	**35%**	Conversation, mid-flow pallet change with visible diff, multi-error uploads handled honestly, carrier reasoning table, no login
Deck communication	10%	Exactly 8 slides, one diagram on slide 5
Live presentation	20%	Demo 1 and Demo 2 rehearsed to the second (Part D)

HARD CONSTRAINTS FROM THE GUIDELINES

- Exactly **3 submission items**: 1-minute demo video link, MVP link, 8-slide deck.
- MVP link must stay live and **work without a login, password or your help until Wednesday 28 October 2026, end of day IST**. A paywall, rate limit or expired session counts as "not working".
- Test data: UK to Germany, France or Netherlands only; GBP only; fixed rate card (Sheet 3) and fixed 10-rule rulebook (Sheet 4).
- The files we have do not state the Stage 1 submission deadline. **Find it on the submission form today** and work backwards from it.

Your 3-day timeline at a glance

When	Goal	Outputs by end of day
Sun 25 Oct (today)	Foundations	Accounts and repo ready; all 21 fixtures with expected outputs; engine started (Person A); parser started (Person B)
Mon 26 Oct	Core product	Engine passes every fixture; parser survives odd files; UI works end to end with zero model calls
Tue 27 Oct	Ship	Gemini layer + fallback; deployed link verified on a phone; dry run; video recorded; deck finished; form submitted if allowed
Wed 28 Oct	Hold the line	Morning re-test of the live link, then do not touch it. Link must work through end of day IST

Total effort is roughly 20-22 person-hours, split between two people working in parallel. If you are behind at any checkpoint, use the cut list in Part E, never the "never cut" list.

PART A - The final solution (what the app does)

A1. Booking Card: the six outputs

1. **Verdict.** **BLOCKED** (any Red) / **SHIP WITH CARE** (Ambers only) / **CLEAR TO SHIP** / *INCOMPLETE* (data gate triggered, see A4).
2. **Findings.** Rule ID, plain-English problem, severity, the numbers involved, suggested fix with confidence label, and an Apply button where safe.
3. **Carrier recommendation.** Chosen option, total cost (price per pallet × pallets), Ready -> Collection -> ETA dates, on-time %, why it fits the seller's priority, the runner-up and the trade-off in one line, and the normalised score table for Balanced.
4. **Confirmation.** The shipment described back in a seller's words, including any fixes applied.
5. **What happens next.** Appointment (if warehouse), documents needed, carton labelling, collection date.
6. **Downloads.** Corrected .xlsx, pre-flight report, and the invoice draft (P2).

A change-log strip sits at the top of the card (A9), and the Assumptions panel (A13) is always one click away.

A2. Architecture

```
Browser (Next.js 14 + TypeScript + Tailwind)
 |- SheetJS parses .xlsx locally (nothing uploaded)
 |- /lib  rulebook, trust, validate, repair, carrier, parse, explain
 |        (pure functions, all unit-tested, zero network)
 |
 '- two serverless routes on Vercel (hold the Gemini key):
      /api/chat     -> returns a PATCH of seller-supplied fields only
      /api/explain  -> ONE batched call per upload, written narrative
Fallback: Local-only mode = templated text, no model calls
```

A3. Field trust model (the backbone)

Mutable: can be wrong, can be proposed for correction	Trusted: never auto-corrected
`booking.pallets`, `booking.cartons`, `booking.declared_value`, `booking.gross_weight`, `item.hs_code`, `item.quantity`, `item.cartons`	`item.unit_price`, `item.units_per_carton`, `item.unit_weight`, and all route, date, priority and destination-type fields

This is why the repair engine is provable rather than a guess: with `units_per_carton` trusted, R-03 has only two unknowns, and the other identities disambiguate them.

A4. Data completeness gate (D-codes): runs before any rule, never emits a Red

Code	Case	Behaviour
D-01	Booking with zero item rows	Status Incomplete: "R-02, R-03, R-04, R-07 could not be checked"
D-02	Item missing a required number	Excluded from sums, stated explicitly, affected rules marked Not checked
D-03	Items row with no parent booking	Listed in an Unmatched section
D-04	Duplicate Booking ID	Both shown and flagged
D-05	Pallets zero or blank	R-05 reported as "cannot compute"
D-06	Non-numeric or negative where a number is expected	Value quarantined, D-code shown

Blank HS code and blank country of origin are **not** D-codes: the rulebook already covers them (R-01 Red, R-09 Amber). Every run ends with a reconciliation line, for example "52 rows read, 48 validated, 2 unmatched, 2 incomplete".

A5. The ten rules: exactly how each is implemented

Sheet 4 is the only source of Reds and Ambers. Our own heuristics may only appear in a separate "FYI, not a rule" section and never change the verdict.

Rule	Severity	Check	Engine notes
R-01	**Red**	Every item HS code has at least 6 digits	Read as text, strip ".0". If 5 digits, add caveat: "may be a lost leading zero, confirm at gov.uk/trade-tariff". Still Red
R-02	**Red**	Declared value = sum of quantity × unit price, within 1%	Tolerance measured against the computed sum (stated assumption). Fix candidate: declared value, High if unique
R-03	**Red**	Per item: units per carton × cartons = quantity	Fix candidates: quantity or item cartons (units per carton is trusted)
R-04	**Red**	Booking cartons = sum of item cartons	Fix candidate: booking cartons
R-05	**Red**	Gross weight ÷ pallets within the *chosen* carrier's cap	See A7 for the three states
R-06	**Red**	No lithium batteries, aerosols, flammable liquids	Explicit = Blocked. Implied = Blocked pending seller confirmation. Never Amber (A6)
R-07	**Amber**	Gross weight at least sum of unit weight × quantity	The fix is a floor: "at least X kg", never High confidence
R-08	**Amber**	Warehouse destination needs delivery appointment	Next step offered: book appointment before collection
R-09	**Amber**	Every item has a country of origin	Ask seller; list allowed countries
R-10	**Amber**	Ready date not in the past	Europe/London date; today passes (A13)

A6. R-06 hazard handling

Code	Trigger	Shown as
R-06-E	Description explicitly names a lithium battery, aerosol or flammable liquid (keyword list)	**RED - Blocked**
R-06-I	Product class normally contains one: wireless earbuds, Bluetooth speaker, cordless vacuum, nail polish, hand sanitiser, and similar (Gemini classifier, output constrained to explicit / implied / none)	**RED - Blocked pending confirmation**, with a one-click seller declaration

Severity never leaves Red; the judgement shows up in the resolution path. The declaration logs "*Confirmed by seller, [date, time] (Europe/London)*". There is no login, so we claim no identity. Local-only mode runs the keyword list plus a built-in list of common implied-hazard product names, so R-06 still works without Gemini.

A7. Carrier engine

Carrier	Service	Transit	On-time	Max kg/pallet	£ per pallet DE / FR / NL
A	Road economy	5 days	92%	**500**	180 / 170 / 160
B	Road standard	3 days	95%	**750**	220 / 210 / 200
C	Road express	2 days	98%	**400**	340 / 330 / 320

Transit, on-time and weight caps are the same on all three lanes; only price differs. Load these from the fixed Sheet 3, not from hardcoded values, so a changed template does not break the app.

Ranking

Filter to the destination lane, then to feasible carriers (kg per pallet at or below the cap; the cap is inclusive), then rank by the seller's priority:

- **Cheapest:** total cost, tie-break transit, then on-time
- **Fastest:** transit, tie-break on-time, then cost
- **Most reliable:** on-time rate, tie-break transit, then cost
- **Balanced:** min-max normalise each attribute across the feasible set (0 = best), then

```
score = 0.40 * cost_n + 0.30 * transit_n + 0.30 * reliability_n     lowest wins
if max == min for an attribute, that term is 0
```

Weights are shown on screen with a slider. Justification: cost is the main concern for SME exporters; a late arrival and a missed delivery appointment cost about the same, so speed and reliability weigh equally. *Worked example, 3 pallets to Germany:* costs 540 / 660 / 1,020 give A 0.60, B 0.35, C 0.40, so Balanced picks Carrier B.

R-05 and "chosen carrier"

The chosen carrier is the recommended one, unless the seller picks another. R-05 then has three outcomes:

State	Condition	Shown as
Pass	Chosen carrier's cap is at least gross weight ÷ pallets	Pass
Red: no carrier fits	Weight per pallet is above every cap on the lane (over 750 kg)	Red, with minimum pallets needed per carrier
Red: seller override	Seller picked a carrier whose cap is exceeded	Red, with "switch to B" or "add N pallets"

Repair hook: pallets is mutable, so the engine proposes `pallets = ceil(gross / cap)` for each carrier. For 1,600 kg that is A: 4, B: 3, C: 4. Gross weight is a second candidate at confidence Possible only, because overstated and genuinely heavy look the same.

The two demo bookings (verified arithmetic)

Booking	Pallets	kg / pallet	Feasible carriers	R-05 / result
Demo 1: R-05 flip 1,600 kg	2	800	none	**Red (no carrier fits)**
	3	533	B only	**Pass, B at £660 to Germany**
	4	400	A, B, C	Pass, all open (C cap is inclusive)
Demo 2: carrier upgrade 1,260 kg, priority Fastest	3	420	A, B	Pass, B chosen (3 days, £660)
	4	315	A, B, C	Pass, C chosen (2 days, £1,360): +£700, 1 day faster

The template's BK-001 (620 kg, 3 pallets) is feasible on every carrier, so adding a pallet to it flips nothing. Do not use it for the "change a detail" moment.

A8. Conversation design

Never ask for these (inferred)

Field	Inferred from
Origin country	Always United Kingdom
Destination country	City: Berlin, Frankfurt, Hamburg = Germany; Paris, Lyon = France; Amsterdam, Rotterdam = Netherlands
Currency	Always GBP
Destination type	"warehouse / DC / fulfilment centre" = Warehouse; "shop / store / office" = Business address; otherwise ask
Ready date	Relative phrases ("next Tuesday") converted to an absolute date, then confirmed

Computed vs asked

Field	Policy
`item.cartons`	Compute only if quantity divides evenly by units per carton; show the arithmetic and ask to confirm. If it does not divide evenly, see the rule below
`booking.cartons`	Compute as the sum of item cartons, confirm
`declared_value`	Compute as the sum of quantity × unit price, confirm
`gross_weight`	**Ask.** Packaging weight is unknowable; offer the goods-only weight as an anchor
`pallets`	**Ask**

> **RULE: THE APP MUST NEVER AUTHOR AN R-03 ERROR**
>
> If quantity does not divide evenly, say so before computing anything. Example for 1,000 units at 12 per carton: "*1,000 doesn't divide into 12s; the last carton would hold 4 units, and the rulebook needs an exact match. Which is right? (a) 996 units in 83 cartons, (b) 1,008 units in 84 cartons, (c) keep 1,000 as 83 cartons of 12 plus 1 carton of 4, booked as a second line for the same SKU.*" Test: every value the app suggests is run through the validator and must pass.

Seller override always wins and is then rule-checked. If a seller insists on numbers that conflict, we accept them and raise the rule. Catching the seller's own inconsistency is the product.

Turn structure: at most 3 questions per turn, grouped as (1) route and timing, (2) what is in the shipment, per item, (3) packing and weight, (4) priority and appointment. The Booking Card sits beside the chat with filled fields solid and missing fields outlined.

The model returns a patch, never the booking

```
{
  "set":   { "pallets": { "value": 4, "source": "add another pallet" } },
  "items": [ { "op": "update", "sku": "MUG-01",
               "fields": { "quantity": { "value": 1080, "source": "1,080 mugs" } } } ],
  "questions_answered": ["pallets"]
}
```

- **Our code merges** the patch. The model never sees or returns the full booking.

- **Source check:** every value carries the phrase it came from; if that phrase is not in the latest user message, the field is rejected.
- **Allow-list:** the model may only touch seller-supplied fields. Inferred fields (countries, currency) and computed fields (cartons, declared value) are written by code only.
- **Change log** is built from the applied patch, so it lists exactly what the seller changed.

A9. Repair engine and confidence

Stage 1: single-field repair. For each mutable field, compute the value implied by the trusted fields and the other identities. Accept only if it clears every failing rule. Score by (rules fixed minus rules broken, then smallest relative change).

> **WORKED CASE**
>
> `quantity=1000, units_per_carton=12, cartons=90, unit_price=5, declared_value=5400`. R-03 fails (12 × 90 = 1,080) and R-02 fails (1,000 × 5 = 5,000). Fixing quantity to 1,080 clears both. Fixing cartons to 83.33 is non-integer, leaves R-02 broken and breaks R-04. Report quantity, confidence High, with both lines of arithmetic.

Stage 2: bounded pair search. If no single fix clears everything, try one booking-level plus one item-level field, capped at 200 combinations; accept only a pair that clears everything.

Stage 3: honest fallback. If nothing clears everything, report each broken rule separately with its most likely field at confidence Possible, auto-apply disabled: "*Two or more numbers conflict and we can't tell which is wrong. Please confirm the actual carton count and quantity.*" Judges will plant multi-error bookings; admitting uncertainty beats blaming the wrong field.

Field type	High confidence requires
Counts (quantity, cartons, pallets)	Unique candidate, clears all failures, every alternative breaks a passing rule, and the value is an integer
Money (declared value)	Same as counts but rounded to 2 decimal places; no integer test
Weight (gross weight)	Never High. R-07 gives a floor, not a value: "must be at least 540 kg, the goods alone, before cartons and pallets". Shown as Possible with the minimum stated

Labels: **High** (apply button enabled), **Possible** (explains the failure, no corroboration, apply disabled), **Ambiguous** (two candidates tie: show both, apply neither).

A10. Change log

Any mid-conversation edit re-runs validation and renders a diff strip at the top of the card, above the fold. Every edit is an append-only timestamped event.

```
Demo 1   Pallets 2 -> 3     Weight/pallet 800 -> 533 kg
         R-05  Red (no carrier fits) -> Pass
         Carrier  none -> B (3d, £660)

Demo 2   Pallets 3 -> 4     Weight/pallet 420 -> 315 kg
         No checks changed
         Carrier  B (3d, £660) -> C (2d, £1,360)    +£700, 1 day faster
```

"No checks changed" in Demo 2 is deliberate: it shows judges we do not invent a flip.

A11. Dates: collection day and ETA

- **Collection** = ready date if Monday-Friday, otherwise the next Monday.
- **ETA** = collection + transit (calendar days), rolled to the next Monday if it lands on a weekend.
- The card shows all three dates: Ready, Collection, ETA. UK bank holidays are out of scope and listed as an assumption.

Case	Ready	Collection	Carrier A (5 days) ETA
BK-001	Thu 5 Nov 2026	Thu 5 Nov	Tue 10 Nov
Weekend case	Sat 7 Nov 2026	Mon 9 Nov	Sat 14 Nov, rolled to **Mon 16 Nov** (4 days later than the naive Thu 12 Nov)

A12. Privacy, stated truthfully, plus Local-only mode

> "Validation runs entirely in your browser. No file is uploaded or stored. Only item descriptions and the computed findings are sent to the model, for the hazard check and the written explanation. Press *Show what was sent* to see exactly what."

Local-only mode is a toggle that disables every model call and uses templated explanations. One feature, three jobs: it makes the privacy claim literally true, it is the automatic fallback on a 429 or missing key (with a visible banner), and it is the demo that still works if Vercel or Google has a bad morning.

A13. Assumptions panel (visible in the UI)

- Timezone **Europe/London** for all date logic, shown on screen. R-10 fails only for dates before today; **today itself passes**.

- Transit is calendar days; collection and ETA follow A11. Weekend handling flagged for the seller to confirm.
- R-02 tolerance of 1% is measured against the computed sum.
- Balanced weights are 40 / 30 / 30 and adjustable.
- Rate card and rulebook are the fixed Sheets 3 and 4.

A14. Commercial invoice draft (priority P2, has a kill switch)

Labelled **"Commercial invoice - draft, 8 of 13 customs fields populated"**. Missing fields render as highlighted blanks: buyer name and address, EORI/VAT, invoice number and date, incoterms, payment terms, signature. The gap is itself a finding: the booking sheet does not carry everything customs needs, a root cause of the holds we exist to prevent. If you are behind on Tuesday, convert it to a "customs readiness checklist" and move on.

PART B - Step-by-step execution plan (everything you need to do)

Roles. Person A = engine and Gemini (logic-heavy). Person B = parser and UI (interface-heavy). Phase 0 is done together. Tick each box as you go.

Phase 0 - Set up and lock the contract

Both · Sun, ~1.5 h

- ☐ Get a Gemini API key from Google AI Studio. Check the model name and daily quota shown for your project (free-tier numbers are not guaranteed and change). Keep the model name in an environment variable so you can swap Flash for Flash-Lite in one line.
- ☐ Create a GitHub repo, add both teammates, and create a free Vercel account linked to it.
- ☐ Scaffold Next.js 14 + TypeScript + Tailwind; add SheetJS, Zod, Vitest. Commit.
- ☐ Set up Claude Code in the repo and create a short `CLAUDE.md` stating the principle (deterministic truth, AI voice), the field trust model (A3), and "no rule logic outside rulebook.ts".
- ☐ Find the Stage 1 submission deadline on the form and write it at the top of the repo README.
- ☐ Write `fixtures/` and `fixtures/expected.json` (table in Part C). Agree on every expected output together.

> **Done when:** repo builds, an empty deploy is live on a Vercel URL, and the expected-output file exists and you both agree on it. After this, everything is making tests pass.

Phase 1 - Engine

Person A · Sun-Mon, ~5 h

- ☐ `rulebook.ts`: ten rules as data objects `{id, severity, text, check(ctx)}`. Load carrier rate card from data, not constants.
- ☐ `trust.ts`: mutable vs trusted field registry (A3).
- ☐ `validate.ts`: D-gate first (A4), then rules, then verdict (Blocked / Ship with care / Clear / Incomplete).
- ☐ `carrier.ts`: feasibility filter, four ranking modes, Balanced formula, `chosenCarrier`, three R-05 states, per-carrier minimum-pallet suggestions, collection and ETA dates (A7, A11).
- ☐ `repair.ts`: three stages and the confidence rules by field type (A9).
- ☐ Add the invariant test: every app-suggested value passes the validator.

> **Done when:** `npm test` passes all file fixtures, including the Stage 3 case reporting separately instead of guessing, and Demo 1 and Demo 2 produce exactly the numbers in A7.

Phase 2 - Parser

Person B · Sun-Mon, ~3 h, in parallel

- ☐ `parse.ts`: find the header row by matching column names (never by position); skip the grey example row; read HS codes as text; parse dates as DD-MM-YYYY; coerce numbers and quarantine bad values (D-06).
- ☐ Link Items to Bookings; surface orphans (D-03) and duplicates (D-04); produce the reconciliation line.
- ☐ Make column order irrelevant and tolerate extra blank rows.

> **Done when:** it survives the official template, a copy with the header moved down two rows, one with reordered columns, and one with a deliberately corrupt row.

Phase 3 - UI, no AI yet

Person B · Mon, ~5 h

- ☐ Layout: chat pane left, Booking Card right (filled fields solid, missing outlined); file drop zone; "Load sample file" button; three one-click example prompts.
- ☐ Booking Card: verdict badge, findings table with Apply buttons and confidence labels, carrier panel with the normalised score table, confirmation paragraph, next-steps checklist.
- ☐ Change-log strip at the top, above the fold (A10).
- ☐ Assumptions panel and Local-only toggle (A12, A13).
- ☐ Multi-booking view for uploads: list with verdict badges, click to open a card, reconciliation line at the top.
- ☐ Templated explanations in `explain.ts` so every finding already reads in plain English with no model.

> **Done when:** the whole app works end to end with zero model calls, including Demo 1 and Demo 2 with a visible change log.

Phase 4 - Gemini layer

Person A · Tue morning, ~3 h

- ☐ `/api/chat`: slot-filling with a Zod schema and Gemini structured output returning the **patch** format (A8). Put the inference table and computed-vs-asked policy in the system prompt. Merge in code; enforce source check and allow-list.
- ☐ `/api/explain`: one batched call per upload for the narrative, never one per rule.
- ☐ R-06 classifier: keyword list first, Gemini only for implied hazards, output constrained to explicit / implied / none.
- ☐ 429, timeout and missing-key guard: automatic switch to Local-only mode with a visible banner, never a stack trace.
- ☐ "Show what was sent" panel listing exactly the text sent to the model.
- ☐ Test the non-divisible-quantity scenario, a seller override onto an infeasible carrier, and a mid-conversation change, using the real model.

> **Done when:** removing the API key entirely degrades the app gracefully, and no chat test lets the model change a field the seller did not mention.

Phase 5 - Deploy and harden

Both · Tue, ~2 h

- ☐ Deploy to Vercel with the key as an environment variable (server side only).
- ☐ Open the link in a private window, on a phone, and on a different network: no login, no key prompt, no cold-start screen.
- ☐ Run the full judge dry run from Part C, once as a stranger would, with the two of you alternating roles.
- ☐ Fix only what the dry run breaks. Freeze features after this step.

> **Done when:** a person who has never seen the app can complete both test parts from the link alone.

Phase 6 - Submission assets

Both · Tue, ~4 h

- ☐ Record the 1-minute video on the deployed URL, not localhost (script in Part D).
- ☐ Build the 8-slide deck (slide-by-slide content in Part D). Title slide must carry both names, the case, the MVP link and the video link.
- ☐ Submit the form if the deadline allows, or schedule it. Double-check the three items open without login.
- ☐ Wednesday morning: re-test the live link end to end. Then do not redeploy. The link must work until end of day IST.

PART C - Test plan

C1. Fixture list (build these in Phase 0)

#	Fixture	Expected outcome
1-10	One booking per rule, each breaking only that rule (R-01 to R-10)	Exactly that rule fires, correct severity, correct verdict
11-13	Three bookings with two planted mistakes each	Stage 2 pair fix if unique, otherwise Stage 3 separate reports at Possible. Never a confident wrong blame
14	Booking with zero item rows	D-01, Incomplete, affected rules "not checked"
15	Items row with no parent booking	D-03, listed under Unmatched
16	Booking with zero pallets	D-05, R-05 "cannot compute", no crash
17	Clean booking	Clear to ship, no findings
18	**Demo 1**: 1,600 kg, 2 pallets	R-05 Red (no carrier fits); at 3 pallets Pass with B at £660
19	**Demo 2**: 1,260 kg, 3 pallets, Fastest	B chosen; at 4 pallets C chosen, +£700, no check flips
20	Seller override onto Carrier C at 500 kg/pallet	R-05 Red (seller override) with "switch to B" or "add pallets"
21	Weekend ready date (Sat 7 Nov 2026)	Collection Mon 9 Nov; Carrier A ETA Mon 16 Nov

Plus chat-only scripts: non-divisible quantity (1,000 units at 12 per carton), an implied hazard ("wireless earbuds"), and a field the seller never mentions staying untouched after a patch.

C2. Judge dry run (mirrors the official test)

Part	What to do, and what must happen
Part 1: conversation	Describe a shipment in one casual sentence, leaving details out. The app asks only for what is missing, in grouped questions, and ends with a complete booking. Then change a detail (add a pallet): the card updates and the change log shows before and after and which checks flipped.
Part 2: file upload	Upload a file with several bookings, some with planted mistakes. Every booking shows what is wrong, how serious, and how to fix it. Multi-error bookings are handled honestly. Reconciliation line accounts for every row.
For every booking	Recommended carrier for the seller's priority, with the reason; a confirmation a seller can understand; what happens next.

C3. Final link checklist (run Tuesday and Wednesday morning)

- ☐ Opens in a private window with no login and no password.
- ☐ Works on a phone and on a different network.
- ☐ Sample file loads in one click; all three example prompts work.
- ☐ Local-only mode works with the Gemini key removed.
- ☐ A file with 1 booking and a file with 50 both work without a timeout.
- ☐ Dates use the live current date.
- ☐ No stack trace or raw error is ever shown.

PART D - Video, deck and live presentation

D1. One-minute demo video script

Time	On screen	Say
0:00-0:10	Title and a blocked card	"Sellers find booking mistakes days later, when stock is stuck at customs. PreFlight catches them before the shipment leaves."
0:10-0:30	Chat: Demo 1 described casually; app asks two grouped questions; then "add a pallet" with the change log flipping Red to Pass	"Tell it what you are shipping. It asks only what is missing. Change one detail and you see exactly what changed."
0:30-0:50	File upload: multi-error booking handled; findings with fixes and confidence labels; reconciliation line	"Upload your sheets. Every booking shows what is wrong, how serious, and the fix. When two numbers conflict, it says so instead of guessing."
0:50-1:00	Carrier panel with the score table	"Then it recommends a carrier for your priority and shows its working."

Record on the deployed URL; rehearse twice; keep it under 60 seconds.

D2. The 8-slide deck, slide by slide

#	Slide	Content to put on it (source in this document)
1	Title	Both names, Case 2, MVP link, video link
2	Problem breakdown	Who has the problem (seller, forwarder ops desk, receiving warehouse), the cost model `P(error) × [hold days × storage + ops touches × cost + P(stockout) × margin]` with how you would verify each term, and the five sub-problems: Capture, Detect, Diagnose and repair, Decide, Commit
3	Scope	In: conversational capture, 10 rules, repair with confidence, carrier recommendation, confirmation, upload. Out with reasons: real carrier booking (no sandbox, fixed rate card), payments, HS classification database, labels, tracking, multi-currency, non-UK origins
4	Assumptions & data	Field trust model, D-gate, R-05 chosen carrier, Balanced weights, Europe/London dates, calendar-day transit; each with how to check it in the real world. Include "the booking sheet lacks fields customs needs"
5	Solution & MVP	One diagram: input (chat or file) -> parser -> data gate -> rules -> repair -> carrier engine -> Booking Card; Gemini shown only as patch, hazard check and explanation; Local-only fallback
6	Adoption	Day-to-day user: the forwarder ops desk and the seller. What changes: from correcting by email days later to clean-or-exception review at booking. Why they might resist: trust in AI numbers, extra step, fear of being blocked. Answer: every number shows its arithmetic; confidence labels; seller override always wins
7	Rollout plan	Pilot with one forwarder's ops desk on one lane; phases (shadow mode, then assisted, then default); what happens when the tool is wrong (override logged, rule reviewed); metrics: share of bookings amended after submission, customs holds per 100 bookings, ops touches per booking, % of bookings opened with the tool (proves usage)
8	Open slide (brownie point)	Pick one: next features (plain-English rule authoring with human approval, bulk fix across bookings, ops queue), or challenges at scale (rule maintenance per lane, HS accuracy), or running cost at volume (batched model calls, local validation keep it near zero)

D3. Live presentation prep (top 10 per section present)

- ☐ Rehearse Demo 1 (Red flips to Pass) and Demo 2 (carrier upgrade, "no checks changed") until they take under 90 seconds.
- ☐ Prepare answers to four likely questions: Why not let the AI do the maths? What if Gemini is down? How do you know your repair is right? How would a forwarder add its own rule?
- ☐ Have Local-only mode ready as the backup demo, and the sample file one click away.

PART E - Risks, cut list and open items

Risk	What would happen	Mitigation
Gemini free-tier limit or outage	429 mid-demo	Batched calls, automatic Local-only fallback with banner
Model changes a field the seller did not touch	Untrustworthy change log	Patch-only schema, source check, allow-list
Judge file has a different layout	Parse failure	Header found by name, order-independent, quarantine bad rows
Host sleeps or link expires	Test counts as "not working"	Vercel serverless (no sleep), Wednesday morning re-test, no redeploys after freezing
Confident wrong repair on multi-error booking	Judges score it as a bad fix	Three-stage search, Possible label, apply disabled

> **CUT LIST, IN THIS ORDER, IF TIME RUNS SHORT**
> Invoice draft, then Balanced weight slider, then change-log carrier delta, then Stage 2 pair search (fall straight through to Stage 3).

> **NEVER CUT**
> The data completeness gate, the 429 / Local-only fallback, and the change log. These are what judges test hardest.

Open items to resolve today

- ☐ Find the Stage 1 submission deadline and set an internal deadline 6 hours before it.
- ☐ Confirm the Gemini model name and daily quota in AI Studio for your key.
- ☐ The reviewer's list skipped items 2, 4 and 7. If those were cut off when copied across, paste them in and fold them into this plan.

First Claude Code prompt

```
Read fixtures/expected.json. Build lib/rulebook.ts, lib/trust.ts and
lib/validate.ts in TypeScript so that `npm test` passes every fixture.
Rules are data objects {id, severity, text, check(ctx)} - no rule logic
inline anywhere else. The data-completeness gate runs before any rule and
must never emit a Red. Do not write the UI or any API route yet.
```

Companion to "Case 2 - Problem Breakdown and Solutions". Free-tier limits and hosting behaviour were accurate as of 25 October 2026; re-check before deployment day.